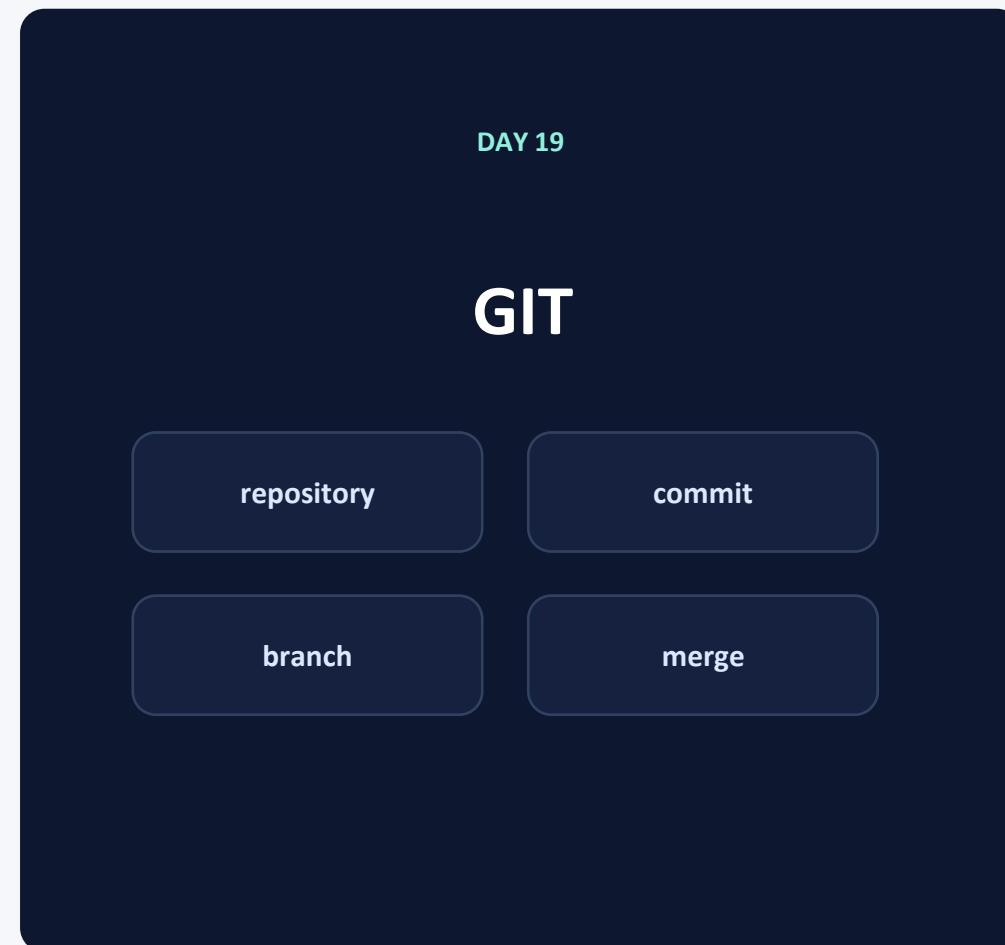


Git и совместная работа

Безопасно хранить историю изменений с Git и работать через рабочую директорию, staging, commit, branch и remote.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Безопасно хранить историю изменений с Git и работать через рабочую директорию, staging, commit, branch и remote.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Объектно-ориентированное программирование

Что уже есть в нашей модели.



СЕГОДНЯ

Git и совместная работа

Вчера: Объектно-ориентированное программирование. Сегодня мы добавляем следующий слой: git и совместная работа.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Git защищает от потери работы, делает изменения объяснимыми и позволяет нескольким людям работать над проектом одновременно.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

**Git — машина времени для проекта:
коммит — сохранённая точка, ветка —
отдельная линия эксперимента, remote
— общая копия для команды.**

ПЕРЕВОД В IT

Git

система контроля версий исходного кода

repository

проект с историей изменений Git

commit

сохранённое осмысленное состояние
изменений

Слой 1: Git • repository • commit

Сначала смысл, затем профессиональный термин.

**Git**

система контроля версий исходного кода

**repository**

проект с историей изменений Git

**commit**

сохранённое осмысленное состояние изменений

Слой 2: branch • merge • remote

Сначала смысл, затем профессиональный термин.

**branch**

отдельная линия разработки

**merge**

объединение изменений из разных веток

**remote**

связанный удалённый репозиторий

Слой 3: working tree • staging area

Сначала смысл, затем профессиональный термин.

working tree

текущие файлы проекта на компьютере

staging area

подготовленный набор изменений для коммита

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: clone • conflict

Сначала смысл, затем профессиональный термин.

clone

создание локальной копии удалённого репозитория

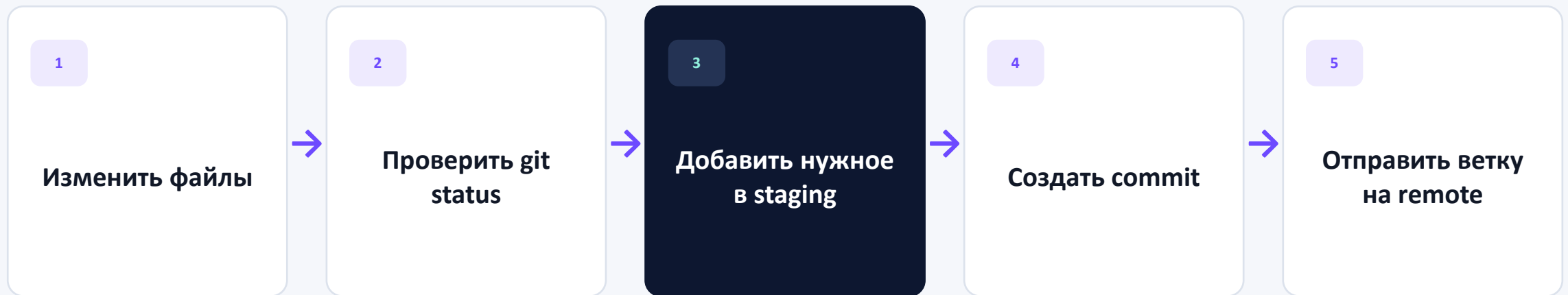
conflict

несовместимые изменения, требующие ручного выбора

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day19_step1

```
git init  
git add app.py  
git commit -m "Add greeting"  
git log --oneline
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

git init

Команда Git изменяет или показывает состояние репозитория.

git add app.py

Команда Git изменяет или показывает состояние репозитория.

git commit -m "Add greeting"

Команда Git изменяет или показывает состояние репозитория.

git log --oneline

Команда Git изменяет или показывает состояние репозитория.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
git init  
git add app.py  
git commit -m "Add greeting"  
git log --oneline
```

РЕЗУЛЬТАТ

Создан репозиторий, app.py
истории одним коммитом

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
git switch -c feature/profile  
git add .  
git commit -m "Add profile page"  
git push -u origin feature/profile
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Новая ветка отправлена на remote

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
git init  
git commit -m "Save app"
```

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

После создания/изменения файла его сначала нужно добавить в staging через git add. Иначе Git не знает, что включать в коммит.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
git init  
git add app.py  
git commit -m "Save app"
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

1

Исчезла ли исходная причина проблемы?

2

Какой результат/поведение теперь ожидаем?

3

Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Исправить баг в отдельной ветке и подготовить изменение к совместной проверке.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Проверяем чистое состояние.**
- 2 Создаём ветку `fix/login`.
- 3 Исправляем файл.
- 4 Добавляем изменения и коммитим.
- 5 Push ветки — после этого можно открыть pull request.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
git status  
git switch -c fix/login  
# изменить app.py  
git add app.py
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
git status
git switch -c fix/login
# изменить app.py
git add app.py
git commit -m "Fix login validation"
git push -u origin fix/login
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

В remote появляется ветка fix/login с отдельным КОММИТОМ

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Какие команды нужны после изменения README.md, чтобы сохранить правку в локальной истории?

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

status → add → commit. push нужен только для удалённого репозитория.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
git status  
git add README.md  
git commit -m "Update README"
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

README.md сохранён в новом локальном коммите

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Собери безопасный поток исправления бага от новой ветки до merge.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
git switch -c fix/bug  
# правка  
git add .  
git commit -m "Fix bug"  
git push -u origin fix/bug  
# затем PR -> review -> merge
```

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Изменение изолировано и готово к review

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Коммитить секреты и .env.

02

Делать огромные коммиты из несвязанных изменений.

03

Игнорировать git status перед commit.

04

Решать конфликт вслепую, не понимая обе версии.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «Git» и какую проблему это понятие помогает описать?

2

Чем «Git» отличается от «branch»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **Git — система контроля версий исходного кода.**

2 Git: система контроля версий исходного кода. branch: отдельная линия разработки.

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Создан репозиторий, app.py сохранён в истории одним коммитом

4 После создания/изменения файла его сначала нужно добавить в staging через git add. Иначе Git не знает, что включать в коммит.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Безопасно хранить историю изменений с Git и работать через рабочую директорию, staging, commit, branch и remote.
- ✓ Читать и объяснять пример темы «Git и совместная работа».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Командная строка и Linux

Завтра: Командная строка и Linux. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.